VS Code+Copilot使用技巧

📂 设置工作区指令文件

关于工作区指令文件，支持以下几种格式：

文件	适用范围
`.github/copilot-instructions.md`	**GitHub Copilot 原生支持**，VS Code 中自动加载
`AGENTS.md`	通用 AI agent 指令，多工具兼容
`CLAUDE.md`	Claude 专用（Claude Code 等）
`.cursorrules`	Cursor 编辑器专用
`.windsurfrules`	Windsurf 编辑器专用

- 如果你主要用 **VS Code + GitHub Copilot**，用 `.github/copilot-instructions.md` 最合适，会被自动识别

⚡ 工作区指令文件 · 使用方法

在 VS Code 中使用 GitHub Copilot Chat，输入：

```
/init
```

或更具体地：

```
/init 生成 AGENTS.md，特别注意：
- 包管理使用 poetry
- 测试框架是 pytest
- 代码风格遵循 PEP 8
```

或者使用 `@workspace` 增强上下文：

```
@workspace /init 生成 AGENTS.md
```

🎯 上下文控制

语法	作用
`#codebase`	让 Copilot 搜索整个代码库回答问题
`#file:pipeline.py`	引用特定文件作为上下文
`#selection`	引用编辑器中选中的代码
`#terminal`	引用终端输出（报错信息等）
`#problems`	引用 VS Code 问题面板中的错误

🛠️ 内置命令

命令	用途
`/init`	自动生成工作区指令文件
`/fix`	修复选中代码或当前文件的问题
`/explain`	解释选中代码
`/tests`	为选中代码生成测试
`/doc`	为选中代码生成文档
`/new`	创建新项目脚手架

💬 高效 Prompt 模式

```
# 结合多个上下文源
#file:pipeline.py #file:generator.p
y 重构这两个文件的公共逻辑

# 限定范围
#codebase 所有使用 lxml 的地方是否正确
处理了命名空间?

# 引用终端报错
#terminal 修复这个错误

# 指定风格
按照 #file:traffic_engineering.py 的
模式，为《计算机学报》创建模板
```

🤖 Agent 模式小技巧

- **多步任务**自动规划：描述目标而非步骤，让 agent 自己拆解
- **运行测试验证**：修改代码后 agent 会自动运行 `poetry run pytest` 验证
- **撤销安全**：所有文件修改都可在编辑器里逐个 Accept/Discard
- **指令文件分层**：可用 `applyTo` 为不同目录设置不同规则（如 tests 单独的测试规范）

🔧 copilot-instructions.md 进阶

可以用 `applyTo` 针对特定文件给不同指令：

```markdown
```

applyTo: "tests/**"

使用 pytest，fixture 放在 conftest.py，测试函数以 test_ 开头。
离线测试时传 use_llm=False。

applyTo: "src/t2t/templates/journals/**"

新模板必须继承 JournalTemplate，用 @register_template 装饰器注册。
参考 TrafficEngineeringTemplate 实现。
```

最大的收益来自**写好 copilot-instructions.md**——它相当于给 Copilot 一份项目专属 prompt，所有对话自动携带。
```